



المدرسة العليا
للأساتذة - مراكش
ÉCOLE NORMALE
SUPÉRIEURE - MARRAKECH

Université Cadi Ayyad
École Normale Supérieure
Département d'Informatique

Licence d'éducation : Spécialité Enseignement Secondaire – Informatique
Semestre S6
Module : Développement Web et Mobile 2

Rapport de Projet de Fin de Module

*Fleet Manager : plateforme web et Android
pour le suivi intelligent d'une flotte de véhicules*

Réalisé par

Wissam Oulad Tohami

Encadré par

Pr. Mohamed LACHGAR

Dépôt du projet : <https://github.com/WissamOuladTohami/Projet-Web-Mobile.git>

Année universitaire : 2025–2026

Déclaration

Je soussigné(e), **Wissam Oulad Tohami**, de l'École Normale Supérieure, Université Cadi Ayyad, Département d'Informatique, déclare que le présent rapport concernant le projet intitulé « *Fleet Manager : plateforme web et Android pour le suivi intelligent d'une flotte de véhicules* », y compris l'ensemble des textes, figures, tableaux, extraits de code et illustrations, constitue le fruit de mon travail personnel. Les sources externes utilisées ont fait l'objet de citations et références explicites. Je reconnais que tout manquement à cet engagement relèverait du plagiat, considéré comme une infraction académique passible de sanctions.

J'autorise la mise à disposition d'un exemplaire de ce rapport à des fins pédagogiques, au bénéfice des futurs étudiants et chercheurs, et j'accepte qu'il soit consulté dans l'intérêt général de l'Enseignement Supérieur et de la Recherche.

Wissam Oulad Tohami

2 mai 2026

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Mes remerciements les plus sincères vont en premier lieu à **Pr. Mohamed LACHGAR**, mon encadrant, pour sa disponibilité, ses précieux conseils, et son accompagnement tout au long de ce module. Sa pédagogie et son expertise en développement web et mobile ont été une source d'inspiration constante.

Je remercie également l'ensemble du corps enseignant du **Département d'Informatique de l'École Normale Supérieure de Marrakech** pour la qualité de la formation dispensée tout au long de mon parcours.

Enfin, je remercie ma famille et mes amis pour leur soutien moral et leur encouragement tout au long de ce travail.

Abstract

This project presents the design and development of a complete vehicle fleet tracking system intended for schools and companies. The solution consists of three complementary components : an Android mobile application dedicated to drivers, a secure REST API developed with Node.js and Express, and a React-based web administration interface built with Vite.

The system allows fleet managers to monitor vehicles in real time on an interactive GPS map (Leaflet), manage drivers and vehicles through a full CRUD interface, track fuel consumption with per-vehicle statistics visualized through bar charts, and visualize vehicle trajectory history with approximate distance calculation. The mobile application enables drivers to start and stop missions, automatically send their GPS position every five seconds, and declare fuel consumption.

The backend API uses JWT authentication (HS256, 24-hour expiry), bcrypt password hashing (cost factor 10), and prepared SQL statements to ensure data security. The database is structured around four tables : *vehicles*, *drivers*, *vehicle_positions*, and *fuel_logs*.

The system was validated through API testing with Postman (16 endpoints), GPS position tracking verified at five-second intervals on an Android emulator, and a fully functional React dashboard confirmed in a local development environment.

Keywords : fleet tracking, GPS geolocation, REST API, Node.js, React, Android, JWT, MySQL, dashboard, fuel consumption.

Résumé

Ce projet présente la conception et le développement d'un système complet de suivi de flotte de véhicules destiné aux écoles et entreprises. La solution est composée de trois composantes complémentaires : une application mobile Android dédiée aux chauffeurs, une API REST sécurisée développée avec Node.js et Express, et une interface web d'administration développée avec React et Vite.

Le système permet aux gestionnaires de flotte de superviser les véhicules en temps réel sur une carte GPS interactive (Leaflet), de gérer les chauffeurs et véhicules via une interface CRUD complète, de suivre les consommations de carburant avec statistiques détaillées par véhicule sous forme de graphiques à barres, et de visualiser l'historique des trajectoires avec calcul de distance approximative. L'application mobile permet aux chauffeurs de démarrer et arrêter leurs missions, d'envoyer automatiquement leur position GPS toutes les cinq secondes, et de déclarer leurs consommations de carburant.

L'API backend utilise l'authentification JWT (algorithme HS256, expiration 24 heures), le hachage des mots de passe avec bcrypt (facteur de coût : 10), et des requêtes SQL préparées pour assurer la sécurité des données. La base de données est structurée autour de quatre tables : *vehicles*, *drivers*, *vehicle_positions* et *fuel_logs*.

Le système présente toutefois des limites : absence de déploiement cloud, précision GPS dépendante des conditions réseau, et tests réalisés uniquement en environnement local.

Mots clés : suivi de flotte, géolocalisation GPS, API REST, Node.js, React, Android, JWT, MySQL, tableau de bord, carburant.

Liste des abréviations

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
GPS	Global Positioning System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

Technologies utilisées : Node.js, Express.js, React, Vite, Android (Java), MySQL, Retrofit2, Leaflet.js, Recharts.

Table des figures

3.1	Architecture generale du systeme de suivi de flotte	18
5.1	Schema Entite-Association de la base de donnees	23
6.1	Dashboard de l'interface web React	32
6.2	Carte GPS avec positions des vehicules	33
6.3	Écran de connexion Android	33

Liste des tableaux

2.1	Acteurs du système, leurs objectifs et droits d'accès	14
4.1	Comparaison des solutions de suivi de flotte	20
5.1	Structure de la table drivers	24
5.2	Structure de la table vehicles	24
5.3	Structure de la table vehicle_positions	24
5.4	Structure de la table fuel_logs	25
5.5	Endpoints de l'API REST	25
6.1	Environnement de développement	27
B.1	Comptes de test par défaut	44

Table des matières

Déclaration	1
Remerciements	2
Abstract	3
Résumé	4
Liste des abréviations	5
Table des figures	6
Liste des tableaux	7
1 Introduction Générale	11
1.1 Contexte	11
1.2 Problématique	11
1.3 Objectifs du Projet	12
1.4 Structure du Rapport	12
2 Contexte et Cahier des Charges	13
2.1 Contexte du Module	13
2.2 Description du Projet	13
2.2.1 Acteurs du Système	13
2.2.2 Structure de la Base de Données	14
2.3 Fonctionnalités Requises	14
2.3.1 Application Mobile Android — Côté Chauffeur	14
2.3.2 Interface Web React — Gestion de Flotte	15
2.4 Règles Métier	15
2.5 Exigences Non Fonctionnelles	15
3 Étude Préliminaire et Technologies	16
3.1 Analyse des Besoins	16

3.1.1	Besoins Fonctionnels	16
3.1.2	Besoins Non Fonctionnels	16
3.2	Technologies Utilisées	16
3.2.1	Backend — Node.js et Express	16
3.2.2	Base de Données — MySQL	17
3.2.3	Interface Web — React et Vite	17
3.2.4	Application Mobile — Android (Java)	18
3.3	Architecture Générale	18
4	État de l'Art	19
4.1	Systèmes de Suivi de Flotte Existants	19
4.1.1	Solutions Commerciales	19
4.1.2	Tableau Comparatif	19
4.1.3	Positionnement de Notre Solution	20
4.2	Technologies Web Modernes	20
4.2.1	APIs REST vs GraphQL	20
4.2.2	Authentification JWT	20
4.2.3	Cartographie Web	21
4.3	Géolocalisation Mobile	21
5	Conception du Système	22
5.1	Architecture du Système	22
5.2	Modèle de la Base de Données	22
5.2.1	Schéma Entité-Association	22
5.2.2	Description des Tables	23
5.3	Conception de l'API REST	25
5.3.1	Endpoints Principaux	25
5.4	Conception de l'Interface Web	26
5.5	Conception de l'Application Android	26
6	Réalisation et Implémentation	27
6.1	Environnement de Développement	27
6.2	Implémentation du Backend	27
6.2.1	Structure du Projet	27
6.2.2	Authentification JWT	28
6.2.3	Sécurisation des Mots de Passe	28
6.2.4	Envoi de Position GPS	29
6.3	Implémentation de l'Interface Web	29
6.3.1	Structure du Projet React	29
6.3.2	Gestion de l'Authentification	30

6.3.3	Carte GPS Interactive	30
6.3.4	Recherche et Filtres	30
6.4	Implémentation de l'Application Android	31
6.4.1	Structure du Projet	31
6.4.2	Envoi GPS Automatique	31
6.4.3	Communication avec l'API	32
6.5	Captures d'Écran	32
6.6	Résultats de Tests	33
7	Conclusion et Perspectives	35
7.1	Bilan du Projet	35
7.2	Difficultés Rencontrées	35
7.2.1	Problèmes Techniques	35
7.3	Perspectives d'Amélioration	36
8	Apport Personnel	37
8.1	Compétences Acquisées	37
8.1.1	Développement Backend	37
8.1.2	Développement Frontend	37
8.1.3	Développement Mobile	37
8.1.4	Gestion de Projet	38
8.2	Réflexion Personnelle	38
A	Code Source — API Backend	40
A.1	Schéma SQL de la Base de Données	40
A.2	Middleware d'Authentification JWT	41
B	Manuel d'Installation et de Déploiement	43
B.1	Prérequis	43
B.2	Installation du Backend	43
B.3	Installation de l'Interface Web	44
B.4	Installation de l'Application Android	44
B.5	Comptes de Test	44

Chapitre 1

Introduction Générale

1.1 Contexte

Dans un monde où la mobilité et la logistique jouent un rôle central dans le fonctionnement des organisations, la gestion efficace d'une flotte de véhicules constitue un enjeu majeur pour les écoles, entreprises et institutions publiques. Selon plusieurs études sectorielles, une mauvaise gestion de flotte peut entraîner une surconsommation de carburant de 15 à 30 % et un allongement significatif des délais de livraison ou de transport. Le suivi en temps réel des véhicules, la gestion des chauffeurs, et le contrôle des consommations de carburant sont des problématiques quotidiennes qui nécessitent des outils numériques adaptés et performants.

Ce projet s'inscrit dans le cadre du module **Développement Web et Mobile 2** du semestre S6, qui vise la réalisation d'applications multi-plateformes orientées services. Il répond à une problématique concrète : comment concevoir un système complet, sécurisé et ergonomique permettant de suivre une flotte de véhicules en temps réel, tout en respectant les contraintes pédagogiques du module ?

1.2 Problématique

Les systèmes de gestion de flotte existants sont souvent complexes, coûteux, et difficiles à adapter aux besoins spécifiques d'une école ou d'une petite entreprise. Il existe donc un besoin réel pour une solution légère, modulaire et accessible, combinant :

- Un suivi GPS en temps réel des véhicules
- Une gestion simplifiée des chauffeurs et affectations
- Un contrôle des consommations de carburant
- Une interface web intuitive pour les gestionnaires
- Une application mobile simple pour les chauffeurs

Le succès du projet sera mesuré selon quatre critères : suivi GPS en temps réel, sécurité des données (JWT + bcrypt), ergonomie de l'interface, et respect de la contrainte de quatre tables maximum.

1.3 Objectifs du Projet

L'objectif principal de ce projet est de développer un système complet de suivi de flotte de véhicules composé de trois composantes :

1. **Une API REST backend** développée avec Node.js et Express, sécurisée par JWT, communiquant avec une base de données MySQL.
2. **Une interface web React** permettant aux gestionnaires de flotte et administrateurs de superviser les véhicules, gérer les chauffeurs, consulter les positions GPS sur une carte interactive, et analyser les consommations de carburant.
3. **Une application Android** dédiée aux chauffeurs, leur permettant de démarrer et arrêter leurs missions, d'envoyer leur position GPS automatiquement, et de déclarer leurs consommations de carburant.

1.4 Structure du Rapport

Ce rapport est organisé comme suit :

- **Chapitre 2** : Contexte et cahier des charges — présente le module, les acteurs, les fonctionnalités requises et les règles métier.
- **Chapitre 3** : Étude préliminaire et technologies — analyse les besoins et justifie les choix technologiques retenus.
- **Chapitre 4** : État de l'art — compare les solutions de suivi de flotte existantes et positionne le projet.
- **Chapitre 5** : Conception du système — détaille l'architecture, la base de données et les diagrammes UML.
- **Chapitre 6** : Réalisation et implémentation — présente le code, les captures d'écran et les résultats de tests.
- **Chapitre 7** : Conclusion et perspectives.
- **Chapitre 8** : Apport personnel.

Chapitre 2

Contexte et Cahier des Charges

2.1 Contexte du Module

Le module Mobile 2 vise la réalisation d'applications multi-plateformes orientées services, composées d'une application Android (client mobile), d'une API backend (Node.js), et d'une interface Web React (administration et supervision). Chaque projet doit respecter une contrainte de simplicité pédagogique : maximum quatre tables dans la base de données, architecture claire, fonctionnalités utiles et réalistes, et interface mobile et web complémentaires.

2.2 Description du Projet

Le projet **Fleet Manager** vise à suivre la position et les informations d'exploitation d'une flotte de véhicules dans un contexte scolaire ou d'entreprise.

2.2.1 Acteurs du Système

Le système implique trois types d'acteurs, décrits dans le tableau [2.1](#).

TABLE 2.1 – Acteurs du système, leurs objectifs et droits d'accès

Acteur	Objectif	Droits	Interface
Chauffeur	Gérer ses missions et envoyer sa position GPS	Lecture véhicule affecté, envoi GPS, déclaration carburant	Android
Gestionnaire	Superviser les véhicules et affecter les chauffeurs	CRUD véhicules/chauffeurs, carte GPS, statistiques	Web React
Administrateur	Gérer l'ensemble du système et les configurations	Tous les droits	Web React

2.2.2 Structure de la Base de Données

La base de données est structurée autour de quatre tables, conformément à la contrainte pédagogique :

1. **vehicles** : informations sur les véhicules (plaque, marque, modèle, statut, chauffeur affecté)
2. **drivers** : informations sur les chauffeurs (nom, email, mot de passe hashé, téléphone, rôle)
3. **vehicle_positions** : historique des positions GPS (latitude, longitude, horodatage, véhicule)
4. **fuel_logs** : journal des consommations de carburant (quantité, note, date, véhicule, chauffeur)

2.3 Fonctionnalités Requises

2.3.1 Application Mobile Android — Côté Chauffeur

- Authentification par email et mot de passe
- Consultation du véhicule affecté
- Démarrage et arrêt de mission
- Envoi automatique de la position GPS toutes les cinq secondes
- Déclaration de carburant consommé

2.3.2 Interface Web React — Gestion de Flotte

- **Gestion de flotte** : CRUD véhicules, CRUD chauffeurs, affectation chauffeur à un véhicule
- **Suivi GPS** : carte interactive, dernière position, historique de trajectoires, véhicules hors ligne
- **Suivi carburant** : enregistrement des consommations, historique, statistiques par véhicule
- **Dashboard** : véhicules actifs, consommation par période, graphiques et compteurs

2.4 Règles Métier

- Une position doit être liée à un véhicule existant.
- Un `fuel_log` doit contenir une quantité et une date.
- Un chauffeur ne peut être actif que sur un seul véhicule à la fois.
- Le statut d'un véhicule passe à *active* lors du démarrage d'une mission et à *offline* lors de son arrêt.
- **GPS indisponible** : l'envoi de position est suspendu et un message d'erreur est affiché au chauffeur.
- **Mission déjà active** : le système bloque le démarrage d'une seconde mission pour le même chauffeur.
- **Chauffeur non affecté** : l'application Android affiche un message d'avertissement et désactive le bouton de démarrage.

2.5 Exigences Non Fonctionnelles

- **Fréquence GPS** : envoi de la position toutes les cinq secondes pendant une mission active.
- **Délai API** : réponse inférieure à 500 ms en environnement local.
- **Sécurité** : tokens JWT avec algorithme HS256 et expiration de 24 heures ; mots de passe hachés avec bcrypt (facteur de coût : 10) ; requêtes SQL préparées.
- **Interface** : validation des formulaires côté client et serveur ; messages d'erreur explicites ; design responsive.
- **Architecture** : code structuré en controllers/routes ; respect de la contrainte quatre tables maximum.

Chapitre 3

Étude Préliminaire et Technologies

3.1 Analyse des Besoins

3.1.1 Besoins Fonctionnels

L'analyse du cahier des charges a permis d'identifier les besoins fonctionnels suivants :

- Authentification sécurisée pour les chauffeurs et administrateurs
- Suivi GPS en temps réel avec envoi périodique des positions
- Gestion complète (CRUD) des véhicules et chauffeurs
- Affectation des chauffeurs aux véhicules avec règle d'unicité
- Suivi et analyse des consommations de carburant
- Visualisation cartographique des positions et trajectoires
- Tableau de bord avec indicateurs et graphiques

3.1.2 Besoins Non Fonctionnels

- **Sécurité** : authentification JWT, hachage des mots de passe
- **Performance** : réponses API rapides, envoi GPS optimisé
- **Ergonomie** : interface intuitive et responsive
- **Maintenabilité** : code structuré et modulaire

3.2 Technologies Utilisées

3.2.1 Backend — Node.js et Express

Node.js (v24.x) est un environnement d'exécution JavaScript côté serveur, basé sur le moteur V8 de Chrome. Il a été choisi pour son architecture événementielle non

bloquante, particulièrement adaptée aux requêtes GPS fréquentes, et sa compatibilité native avec le format JSON.

Express.js est un framework web minimaliste pour Node.js qui simplifie la création d'APIs REST. Il fournit un système de routage, des middlewares, et une gestion simplifiée des requêtes HTTP.

Les bibliothèques utilisées côté backend sont :

- **mysql2** (v3.x) : connecteur MySQL avec support des requêtes préparées
- **jsonwebtoken** (v9.x) : génération et vérification des tokens JWT
- **bcryptjs** (v2.x) : hachage sécurisé des mots de passe
- **express-validator** (v7.x) : validation des données côté serveur
- **dotenv** (v16.x) : gestion des variables d'environnement
- **cors** (v2.x) : gestion des politiques Cross-Origin
- **nodemon** (v3.x) : rechargement automatique en développement

3.2.2 Base de Données — MySQL

MySQL 8.0 est un système de gestion de base de données relationnelle open source, largement utilisé dans les applications web. Dans ce projet, MySQL est exécuté via **XAMPP 8.2**, un environnement de développement local intégrant Apache, MySQL et PHP.

Les requêtes SQL utilisent le système de requêtes préparées (paramètres positionnels ?) pour prévenir les injections SQL.

3.2.3 Interface Web — React et Vite

React (v18.x) est une bibliothèque JavaScript développée par Meta pour la construction d'interfaces utilisateur basées sur des composants réutilisables. Son architecture basée sur les hooks (**useState**, **useEffect**, **useMemo**) permet une gestion efficace de l'état applicatif. React a été préféré à Vue.js ou Angular pour sa large communauté et la richesse de son écosystème de bibliothèques cartographiques.

Vite (v5.x) est un outil de build moderne qui offre un démarrage instantané du serveur de développement et une compilation optimisée pour la production.

Les bibliothèques utilisées côté frontend sont :

- **axios** (v1.x) : client HTTP pour les appels API
- **react-router-dom** (v6.x) : navigation entre les pages
- **leaflet** / **react-leaflet** (v4.x) : affichage de la carte GPS interactive
- **recharts** (v2.x) : graphiques et visualisations de données

3.2.4 Application Mobile — Android (Java)

L'application Android est développée en **Java** avec **Android Studio Hedgehog 2023.1.1**. Elle cible les appareils Android avec un SDK minimum de 24 (Android 7.0).

Les bibliothèques utilisées sont :

- **Retrofit2** (v2.9) : client HTTP type-safe pour Android
- **Gson** (v2.10) : sérialisation/désérialisation JSON
- **OkHttp** (v4.x) : client HTTP bas niveau avec intercepteurs
- **Google Play Services Location** (v21.x) : accès à la position GPS via FusedLocationProvider

3.3 Architecture Générale

L'architecture du système suit le modèle client-serveur avec trois couches distinctes :

1. **Couche présentation** : application Android et interface React
2. **Couche métier** : API REST Node.js avec controllers et routes
3. **Couche données** : base de données MySQL

La communication entre les clients et le serveur se fait exclusivement via des requêtes **HTTP/JSON**, sécurisées par des tokens JWT transmis dans les headers de chaque requête. La figure 3.1 illustre ces flux de données.

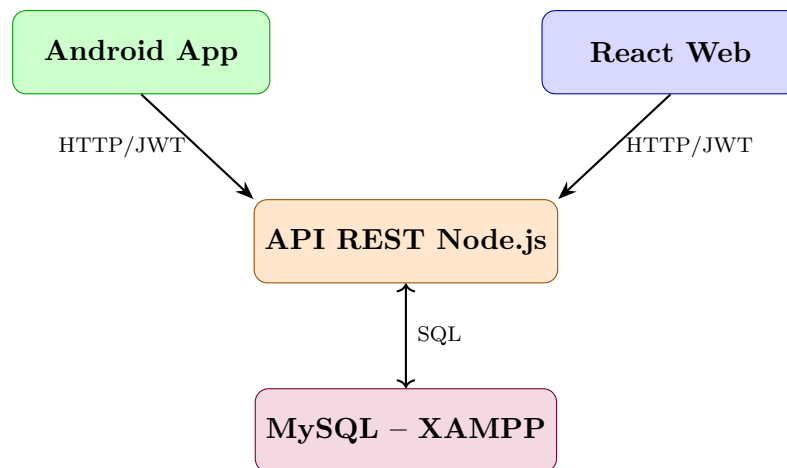


FIGURE 3.1 – Architecture generale du systeme de suivi de flotte

Chapitre 4

État de l'Art

4.1 Systèmes de Suivi de Flotte Existants

Le marché des systèmes de gestion de flotte est dominé par plusieurs solutions commerciales et open source. Cette section présente une analyse comparative des principales solutions existantes et positionne notre projet par rapport à celles-ci.

4.1.1 Solutions Commerciales

Google Fleet Management [12] et **Wialon** [11] sont parmi les plateformes les plus utilisées au niveau mondial. Elles offrent des fonctionnalités avancées telles que le suivi satellite en temps réel, la gestion des alertes, l'optimisation des itinéraires, et la génération de rapports détaillés. Cependant, leur coût élevé et leur complexité les rendent inadaptées aux petites structures comme les écoles.

Traccar [10] est une solution open source de suivi GPS qui supporte de nombreux protocoles de communication. Bien que flexible, elle nécessite une infrastructure serveur dédiée et des compétences techniques avancées pour son déploiement.

4.1.2 Tableau Comparatif

Le tableau 4.1 compare les principales solutions selon six critères pertinents pour ce projet.

TABLE 4.1 – Comparaison des solutions de suivi de flotte

Critère	Google Fleet	Wialon	Traccar	Fleet Manager (ce projet)
GPS temps réel	✓	✓	✓	✓
Open source	✗	✗	✓	✓
Application Android	✓	✓	✓	✓
Dashboard web	✓	✓	✓	✓
Rapports PDF	✓	✓	✓	✗
Déploiement local	✗	✗	✗	✓
Coût	Élevé	Élevé	Gratuit	Gratuit

4.1.3 Positionnement de Notre Solution

Notre système se distingue par :

- **Simplicité** : architecture légère avec quatre tables seulement
- **Accessibilité** : déployable localement avec XAMPP, sans abonnement
- **Complémentarité** : mobile + web dans une architecture unifiée
- **Pédagogie** : code structuré et documenté, idéal pour l'apprentissage

4.2 Technologies Web Modernes

4.2.1 APIs REST vs GraphQL

Les APIs REST (Representational State Transfer) [13] restent la norme dominante pour les services web. Elles utilisent les méthodes HTTP standard (GET, POST, PUT, DELETE) et le format JSON pour l'échange de données. Notre choix s'est porté sur REST pour trois raisons : sa simplicité d'intégration avec Retrofit2 côté Android, sa facilité de test avec Postman, et sa suffisance pour les besoins de ce projet. GraphQL aurait apporté une complexité supplémentaire inutile dans ce contexte pédagogique.

4.2.2 Authentification JWT

Le standard JSON Web Token (JWT) [7] est largement adopté pour l'authentification sans état (stateless). Un token JWT contient trois parties encodées en Base64 : l'en-tête (algorithme), la charge utile (données utilisateur), et la signature. Notre implémentation utilise l'algorithme **HS256** avec une expiration de **24 heures**.

4.2.3 Cartographie Web

Leaflet.js [6] est la bibliothèque de cartographie open source la plus populaire pour le web. Légère (42 KB gzippée), elle supporte de nombreuses sources de tuiles (OpenStreetMap, CartoDB) et offre des fonctionnalités riches : marqueurs, polygones, popups. Notre projet l'utilise pour afficher les positions GPS et les trajectoires des véhicules.

4.3 Géolocalisation Mobile

L'API **FusedLocationProvider** [9] de Google Play Services est la méthode recommandée pour accéder à la position GPS sur Android. Elle combine intelligemment le GPS, le Wi-Fi et les réseaux mobiles pour fournir la position la plus précise possible tout en optimisant la consommation de batterie.

Chapitre 5

Conception du Système

5.1 Architecture du Système

Comme illustré dans la figure [3.1](#) du chapitre précédent, le système est composé de trois couches interconnectées communiquant via une API REST sécurisée par JWT. Les flux de données suivent deux chemins principaux : l'application Android envoie les positions GPS et les logs carburant vers l'API ; l'interface React interroge l'API pour afficher les données en temps réel sur le dashboard et la carte.

5.2 Modèle de la Base de Données

5.2.1 Schéma Entité-Association

La base de données est composée de quatre tables reliées par des clés étrangères, comme le montre la figure [5.1](#).

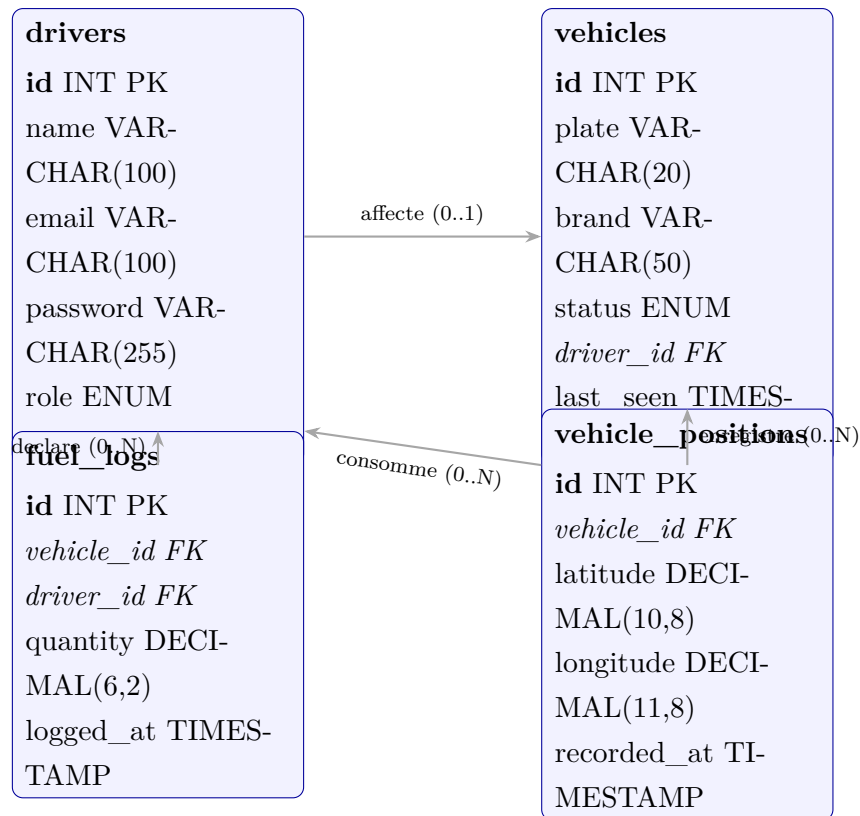


FIGURE 5.1 – Schema Entite-Association de la base de donnees

Les relations sont les suivantes :

- **drivers** → **vehicles** : un chauffeur peut être affecté à zéro ou un véhicule (relation 1..0,1).
- **vehicles** → **vehicle_positions** : un véhicule peut avoir zéro ou plusieurs positions enregistrées (relation 1..0,N).
- **vehicles** → **fuel_logs** : un véhicule peut avoir zéro ou plusieurs logs carburant (relation 1..0,N).
- **drivers** → **fuel_logs** : un chauffeur peut déclarer zéro ou plusieurs consommations (relation 1..0,N).

5.2.2 Description des Tables

Le tableau 5.1 décrit la structure de la table **drivers**.

TABLE 5.1 – Structure de la table drivers

Champ	Type	Contrainte	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Identifiant unique
name	VARCHAR(100)	NOT NULL	Nom complet
email	VARCHAR(100)	NOT NULL, UNIQUE	Adresse email
password	VARCHAR(255)	NOT NULL	Mot de passe hashé
phone	VARCHAR(20)	NULL	Numéro de téléphone
role	ENUM	DEFAULT 'driver'	Rôle (admin/driver)
status	ENUM	DEFAULT 'active'	Statut (active/inactive)
created_at	TIMESTAMP	DEFAULT NOW()	Date de création

Le tableau 5.2 décrit la structure de la table **vehicles**.

TABLE 5.2 – Structure de la table vehicles

Champ	Type	Contrainte	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Identifiant unique
plate	VARCHAR(20)	NOT NULL, UNIQUE	Numéro de plaque
brand	VARCHAR(50)	NULL	Marque du véhicule
model	VARCHAR(50)	NULL	Modèle du véhicule
status	ENUM	DEFAULT 'offline'	Statut du véhicule
driver_id	INT	FOREIGN KEY	Chauffeur affecté
last_seen	TIMESTAMP	NULL	Dernière activité
created_at	TIMESTAMP	DEFAULT NOW()	Date d'ajout

Le tableau 5.3 décrit la structure de la table **vehicle_positions**.

TABLE 5.3 – Structure de la table vehicle_positions

Champ	Type	Contrainte	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Identifiant unique
vehicle_id	INT	NOT NULL, FOREIGN KEY	Véhicule associé
latitude	DECIMAL(10,8)	NOT NULL	Latitude GPS
longitude	DECIMAL(11,8)	NOT NULL	Longitude GPS
recorded_at	TIMESTAMP	DEFAULT NOW()	Horodatage

Le tableau 5.4 décrit la structure de la table **fuel_logs**.

TABLE 5.4 – Structure de la table fuel_logs

Champ	Type	Contrainte	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Identifiant unique
vehicle_id	INT	NOT NULL, FOREIGN KEY	Véhicule associé
driver_id	INT	NULL, FOREIGN KEY	Chauffeur associé
quantity	DECIMAL(6,2)	NOT NULL	Quantité en litres
note	TEXT	NULL	Note optionnelle
logged_at	TIMESTAMP	DEFAULT NOW()	Date d'enregistrement

5.3 Conception de l'API REST

5.3.1 Endpoints Principaux

Le tableau 5.5 liste les seize endpoints de l'API REST avec leurs méthodes, chemins, authentification requise et description.

TABLE 5.5 – Endpoints de l'API REST

Méthode	Endpoint	Auth	Description
POST	/api/auth/login	Non	Authentification
POST	/api/auth/register	Non	Inscription
GET	/api/vehicles	JWT	Liste des véhicules
POST	/api/vehicles	JWT	Créer un véhicule
PUT	/api/vehicles/ :id	JWT	Modifier un véhicule
PUT	/api/vehicles/ :id/assign	JWT	Affecter un chauffeur
DELETE	/api/vehicles/ :id	JWT	Supprimer un véhicule
GET	/api/vehicles/offline	JWT	Véhicules hors ligne
GET	/api/drivers	JWT	Liste des chauffeurs
POST	/api/drivers	JWT	Créer un chauffeur
DELETE	/api/drivers/ :id	JWT	Supprimer un chauffeur
POST	/api/positions	JWT	Envoyer une position
GET	/api/positions/all-last	JWT	Dernières positions
GET	/api/positions/history/ :id	JWT	Historique positions
GET	/api/fuel-logs	JWT	Liste des logs
POST	/api/fuel-logs	JWT	Ajouter un log
GET	/api/fuel-logs/stats	JWT	Statistiques carburant

5.4 Conception de l'Interface Web

L'interface web React est organisée en sept pages principales :

- **Login** : authentification avec design split-screen
- **Dashboard** : tableau de bord avec quatre cartes de statistiques, graphique linéaire GPS, diagramme circulaire des statuts, et graphique à barres des consommations
- **Véhicules** : CRUD avec recherche, filtres et affectation
- **Chauffeurs** : CRUD avec recherche et filtres par rôle
- **Carte GPS** : carte Leaflet avec positions en temps réel
- **Carburant** : historique et statistiques de consommation
- **Historique** : trajectoires et véhicules hors ligne

5.5 Conception de l'Application Android

L'application Android est composée de trois activités :

- **LoginActivity** : écran de connexion avec validation
- **MainActivity** : écran principal avec gestion de mission et envoi GPS automatique
- **FuelActivity** : formulaire de déclaration de carburant

Le flux de navigation suit le schéma : `LoginActivity` $\rightarrow$ `MainActivity` $\leftrightarrow$ `FuelActivity`.

Chapitre 6

Réalisation et Implémentation

6.1 Environnement de Développement

Le tableau 6.1 liste les outils utilisés avec leurs versions exactes.

TABLE 6.1 – Environnement de développement

Outil	Version / Description
Visual Studio Code	v1.89 — éditeur de code principal
Android Studio	Hedgehog 2023.1.1 — IDE Android
XAMPP	v8.2 (MySQL 8.0) — serveur local
Node.js	v24.x — environnement d'exécution
npm	v10.x — gestionnaire de paquets
Postman	v11.x — test des endpoints API
phpMyAdmin	v5.x — administration base de données
Git	v2.x — contrôle de version

6.2 Implémentation du Backend

6.2.1 Structure du Projet

Le projet backend suit une architecture **routes** → **controllers**, chaque couche ayant un rôle précis : les routes définissent les URLs et délèguent aux controllers ; les controllers contiennent la logique métier et interrogent la base de données ; le middleware vérifie l'authentification avant chaque route protégée.

```
1 fleet-api /
2   config /
3     db.js           # Pool de connexions MySQL
4   middleware /
```

```
5   authMiddleware.js   # Verification JWT + protection routes
6   routes/
7     auth.routes.js
8     vehicles.routes.js
9     drivers.routes.js
10    positions.routes.js
11    fuellogs.routes.js
12  controllers/
13    auth.controller.js
14    vehicles.controller.js
15    drivers.controller.js
16    positions.controller.js
17    fuellogs.controller.js
18  .env                  # Variables d'environnement (non versionne)
19  server.js             # Point d'entree de l'application
```

Listing 6.1 – Structure du projet backend

6.2.2 Authentification JWT

L'authentification repose sur le standard JWT [7]. Lors de la connexion, le serveur génère un token signé contenant l'identifiant, le nom et le rôle de l'utilisateur, avec une durée de validité de 24 heures. Ce token est envoyé dans le header **Authorization: Bearer <token>** de chaque requête protégée. En cas d'expiration, le serveur retourne une erreur 403 et le client doit se reconnecter.

```
1  const token = jwt.sign(
2    { id: user.id, role: user.role, name: user.name },
3    process.env.JWT_SECRET,           // cle secrete dans .env
4    { expiresIn: '24h' }
5  );
6  // Token renvoye au client dans la reponse JSON
7  res.json({ token, role: user.role, name: user.name });
```

Listing 6.2 – Génération du token JWT

6.2.3 Sécurisation des Mots de Passe

Les mots de passe sont hachés avec **bcryptjs** avant stockage, avec un facteur de coût de 10 qui garantit un compromis sécurité/performance adapté à ce projet.

```
1  // A l'inscription : hachage avant insertion en base
2  const hashed = await bcrypt.hash(password, 10);
3
4  // A la connexion : comparaison avec le hash stocke
5  const valid = await bcrypt.compare(password, user.password);
```

```
6 if (!valid) return res.status(401).json({ message: 'Identifiants  
    incorrects' });
```

Listing 6.3 – Hachage du mot de passe

6.2.4 Envoi de Position GPS

L'endpoint POST /api/positions reçoit les coordonnées GPS, les enregistre dans vehicle_positions, et met à jour le statut et la date de dernière activité du véhicule atomiquement.

```
1 await db.query(  
2   'INSERT INTO vehicle_positions (vehicle_id, latitude, longitude) VALUES  
    (?, ?, ?)',  
3   [vehicle_id, latitude, longitude]  
4 );  
5 await db.query(  
6   'UPDATE vehicles SET status="active", last_seen=NOW() WHERE id=?',  
7   [vehicle_id]  
8 );  
9 res.json({ message: 'Position enregistree' });
```

Listing 6.4 – Enregistrement de position GPS

6.3 Implémentation de l'Interface Web

6.3.1 Structure du Projet React

```
1 fleet-web/src/  
2   context/  
3     AuthContext.jsx      # Gestion session utilisateur globale  
4   services/  
5     api.js               # Configuration Axios + intercepteur JWT  
6   components/  
7     Layout.jsx           # Sidebar + topbar + outlet React Router  
8   pages/  
9     Login.jsx            # Authentification  
10    Dashboard.jsx         # Tableau de bord interactif  
11    Vehicles.jsx          # CRUD vehicules  
12    Drivers.jsx           # CRUD chauffeurs  
13    MapView.jsx           # Carte GPS Leaflet  
14    FuelLogs.jsx          # Suivi carburant  
15    History.jsx           # Historique trajectoires
```

Listing 6.5 – Structure du projet React

6.3.2 Gestion de l'Authentification

La session est gérée via un Context React combiné au `localStorage` pour la persistance. L'intercepteur Axios injecte automatiquement le token JWT dans chaque requête. En cas d'expiration (erreur 401), l'intercepteur de réponse redirige automatiquement vers la page de connexion.

```
1 // Injection du token dans chaque requete
2 api.interceptors.request.use((config) => {
3   if (authToken)
4     config.headers.Authorization = 'Bearer ${authToken}';
5   return config;
6 });
7
8 // Gestion automatique de l'expiration du token
9 api.interceptors.response.use(
10  (response) => response,
11  (error) => {
12    if (error.response?.status === 401) {
13      localStorage.removeItem('token');
14      window.location.href = '/login';
15    }
16    return Promise.reject(error);
17  }
18 );
```

Listing 6.6 – Intercepteur Axios — injection JWT et gestion expiration

6.3.3 Carte GPS Interactive

La carte GPS utilise **React-Leaflet** [6] avec les tuiles CartoDB Dark pour correspondre au thème sombre de l'interface. Elle affiche en temps réel la dernière position de chaque véhicule et permet de visualiser les trajectoires historiques sous forme de polylignes dorées.

6.3.4 Recherche et Filtres

Les pages Véhicules, Chauffeurs et Carburant intègrent un système de recherche et filtrage en temps réel implémenté avec le hook `useMemo` pour optimiser les performances :

```
1 const filtered = useMemo(() => {
2   let list = [...vehicles];
3   if (search)
4     list = list.filter(v =>
5       v.plate?.toLowerCase().includes(search.toLowerCase()) ||
6       v.brand?.toLowerCase().includes(search.toLowerCase())
7     );
8   return list;
9 });
```

```

7     );
8     if (filterStatus != 'all')
9         list = list.filter(v => v.status === filterStatus);
10    return list;
11 }, [vehicles, search, filterStatus]);

```

Listing 6.7 – Filtrage avec useMemo

6.4 Implémentation de l'Application Android

6.4.1 Structure du Projet

```

1 com.fleet.driver/
2   LoginActivity.java      # Ecran de connexion avec validation
3   MainActivity.java       # Mission + envoi GPS automatique
4   FuelActivity.java       # Declaration de carburant
5   ApiClient.java         # Configuration Retrofit (base URL)
6   ApiService.java        # Interface des endpoints REST
7   SessionManager.java    # Gestion session locale (SharedPreferences)
8   LoginResponse.java     # Modele reponse login
9   Vehicle.java           # Modele vehicule

```

Listing 6.8 – Structure du projet Android

6.4.2 Envoi GPS Automatique

L'envoi de position GPS est implémenté avec un **Handler** Android qui s'exécute toutes les **cinq secondes** pendant une mission active. Les permissions `ACCESS_FINE_LOCATION` et `ACCESS_COARSE_LOCATION` sont demandées au démarrage. En cas d'indisponibilité du GPS, un message est affiché à l'utilisateur et l'envoi est suspendu.

```

1 private Runnable gpsRunnable = new Runnable() {
2     @Override
3     public void run() {
4         if (ActivityCompat.checkSelfPermission(MainActivity.this,
5             Manifest.permission.ACCESS_FINE_LOCATION)
6             == PackageManager.PERMISSION_GRANTED) {
7             sendLocation();
8         } else {
9             showToast("Permission GPS non accordée");
10        }
11        gpsHandler.postDelayed(this, 5000); // 5 secondes
12    }
13 };

```

Listing 6.9 – Envoi GPS périodique — 5 secondes

6.4.3 Communication avec l'API

La communication avec le backend utilise **Retrofit2** [8] avec un intercepteur OkHttpClient pour l'injection automatique du token JWT :

```
1 .addInterceptor(chain -> {  
2     Request original = chain.request();  
3     Request.Builder builder = original.newBuilder();  
4     if (token != null) {  
5         builder.header("Authorization", "Bearer " + token);  
6     }  
7     return chain.proceed(builder.build());  
8 })
```

Listing 6.10 – Intercepteur JWT Android

6.5 Captures d'Écran

La figure 6.1 présente le tableau de bord de l'interface web React, avec les quatre compteurs statistiques, le graphique d'activité GPS linéaire, le diagramme circulaire des statuts de flotte, et le graphique à barres des consommations par véhicule.

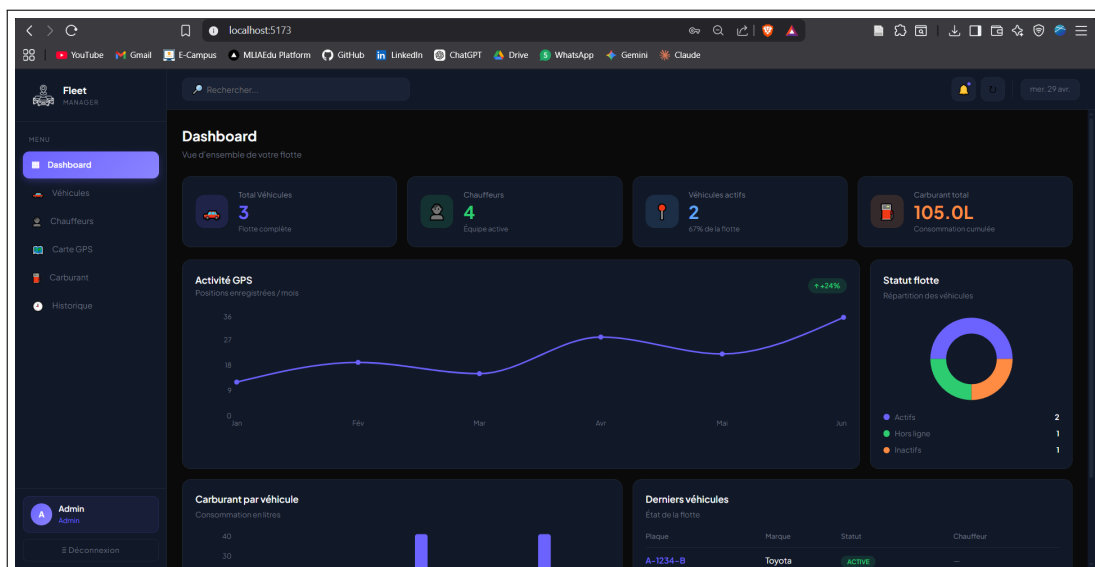


FIGURE 6.1 – Dashboard de l'interface web React

La figure 6.2 montre la carte GPS interactive avec les positions en temps réel des véhicules actifs, affichées sous forme de marqueurs avec informations au survol.

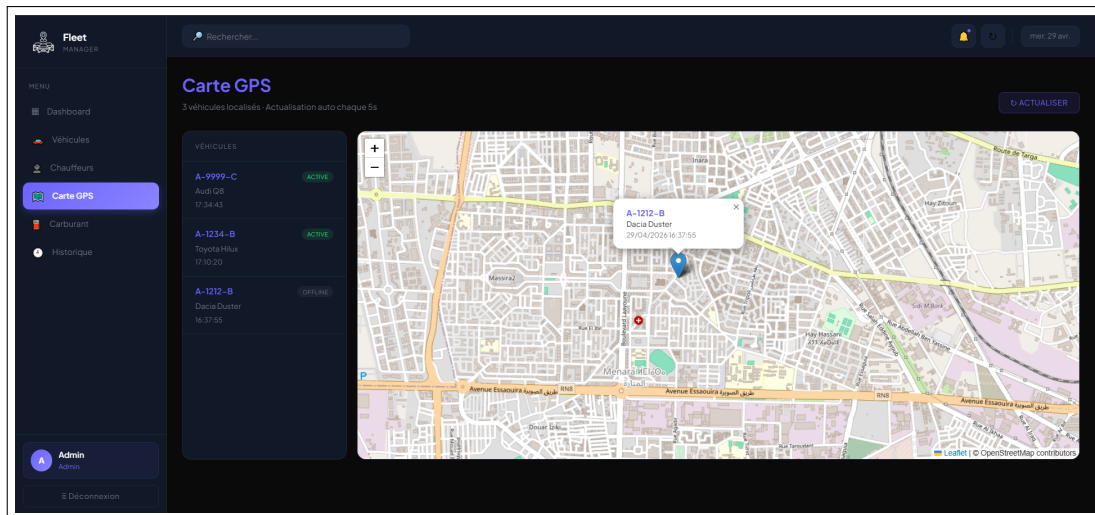


FIGURE 6.2 – Carte GPS avec positions des véhicules

La figure 6.3 présente l'écran de connexion de l'application Android avec validation des champs email et mot de passe.

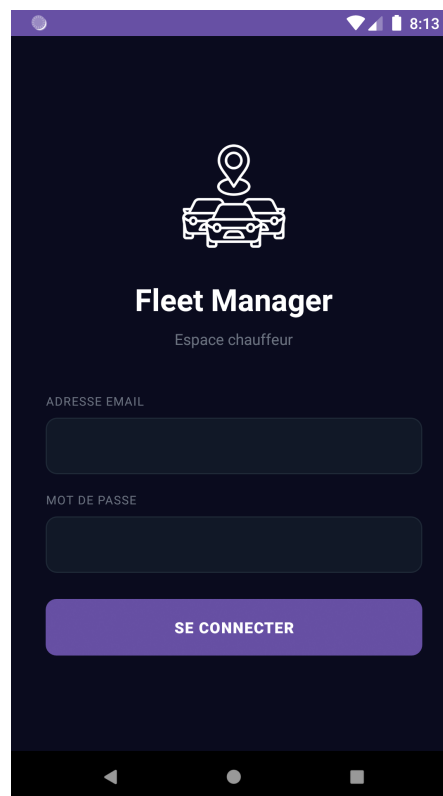


FIGURE 6.3 – Écran de connexion Android

6.6 Résultats de Tests

Le système a été validé selon les critères suivants :

- **API REST** : les 16 endpoints ont été testés avec Postman ; tous retournent les codes HTTP attendus (200, 201, 400, 401, 403, 404).
- **Authentication** : tokens valides acceptés ; tokens expirés ou invalides rejetés avec erreur 403.
- **GPS Android** : envoi de position validé toutes les cinq secondes sur émulateur ; les coordonnées apparaissent en temps réel sur la carte React.
- **Dashboard** : graphiques et compteurs fonctionnels avec données de test (trois véhicules, quatre chauffeurs, vingt logs carburant).
- **Sécurité SQL** : tentatives d'injection SQL neutralisées par les requêtes préparées.

Chapitre 7

Conclusion et Perspectives

7.1 Bilan du Projet

Ce projet a permis de concevoir et développer un système complet de suivi de flotte de véhicules, répondant aux exigences du cahier des charges du module Mobile 2. Les trois composantes du système — l'API REST Node.js, l'interface web React et l'application Android — fonctionnent de manière cohérente et complémentaire.

Les résultats observables sont les suivants :

- **Backend** : 16 endpoints validés sous Postman, authentification JWT opérationnelle, CRUD complet sur véhicules et chauffeurs, gestion des positions GPS et logs carburant.
- **Interface web** : tableau de bord interactif avec quatre graphiques, carte GPS en temps réel, recherche et filtres avancés.
- **Application Android** : login sécurisé, gestion de mission, envoi GPS automatique toutes les cinq secondes, déclaration de carburant.
- **Contraintes** : quatre tables maximum respectées, code structuré en routes/controllers, sécurité des données assurée.

7.2 Difficultés Rencontrées

7.2.1 Problèmes Techniques

- **Chemins avec espaces** : le chemin du projet contenait des espaces et des caractères spéciaux (&), causant des problèmes avec les outils Node.js sous Windows PowerShell. Solution : renommer le dossier sans caractères spéciaux.
- **Politique d'exécution PowerShell** : npm était bloqué par la politique de sécurité PowerShell. Solution : `Set-ExecutionPolicy RemoteSigned`.

- **Persistance de session** : la gestion du token JWT entre le contexte React et les requêtes Axios a nécessité une architecture soignée avec un module de token partagé et un intercepteur de réponse pour gérer l'expiration.

7.3 Perspectives d'Amélioration

Les évolutions prioritaires pour les versions futures sont :

1. **Déploiement cloud** : hébergement de l'API sur un VPS et de la base de données sur un service cloud (PlanetScale, Railway).
2. **Notifications en temps réel** : intégration de WebSocket (Socket.io) pour alerter le gestionnaire en cas de véhicule hors ligne ou de dépassement de zone.
3. **Optimisation GPS** : intégration d'une API de calcul d'itinéraires (OpenRouteService) pour des distances précises.
4. **Alertes carburant** : notifications automatiques lors d'une consommation anormale.
5. **Application iOS** : extension à iOS avec React Native (évolution future, hors scope du module actuel).
6. **Rapports PDF** : génération de rapports exportables avec historique des missions et consommations.

Chapitre 8

Apport Personnel

8.1 Compétences Acquises

8.1.1 Développement Backend

Ce projet a permis de maîtriser le développement d'APIs REST avec Node.js et Express.js. Les compétences acquises incluent : structuration d'un projet selon le pattern routes/controllers, implémentation de l'authentification JWT de bout en bout (génération, vérification, middleware), sécurisation des mots de passe avec bcrypt, et utilisation des requêtes SQL préparées avec mysql2 pour prévenir les injections.

8.1.2 Développement Frontend

Du côté interface web, ce projet a permis d'approfondir la maîtrise de React avec les hooks modernes (`useState`, `useEffect`, `useMemo`, `useContext`). Les compétences acquises incluent : conception d'une architecture de composants réutilisables, gestion de l'état global avec Context API, intégration de Leaflet pour la cartographie et Recharts pour les visualisations de données.

8.1.3 Développement Mobile

Ce projet représente la première expérience complète de développement Android en Java. Les compétences acquises incluent : utilisation de Retrofit2 pour les appels API REST typés, gestion des permissions système (GPS, Internet), accès à la localisation avec FusedLocationProvider [9], et structuration d'une application Android en activités avec cycle de vie.

8.1.4 Gestion de Projet

Ce projet a également permis de développer des compétences en gestion de projet technique : analyse des besoins, conception d’une architecture système multi-tiers, planification des étapes de développement, et résolution méthodique des problèmes techniques rencontrés.

8.2 Réflexion Personnelle

Ce projet constitue une expérience formatrice qui illustre concrètement comment les différentes technologies web et mobile s’articulent pour former un système cohérent. La réalisation d’un projet complet — du backend à l’interface web en passant par l’application mobile — offre une vision globale du développement logiciel moderne orienté services.

La contrainte des quatre tables maximum, loin d’être une limitation, a conduit à concevoir un système simple mais efficace, centré sur les besoins essentiels plutôt que sur la complexité architecturale.

Les compétences acquises lors de ce projet constituent une base solide pour de futurs développements en architecture web et mobile, tant dans le cadre académique que professionnel. Les annexes qui suivent présentent le schéma SQL complet et le manuel d’installation du système.

Bibliographie

- [1] Node.js Foundation, *Node.js Documentation*, <https://nodejs.org/en/docs/>, 2024.
- [2] OpenJS Foundation, *Express.js Documentation*, <https://expressjs.com/>, 2024.
- [3] Meta Open Source, *React Documentation*, <https://react.dev/>, 2024.
- [4] Evan You et al., *Vite Documentation*, <https://vitejs.dev/>, 2024.
- [5] Oracle Corporation, *MySQL 8.0 Reference Manual*, <https://dev.mysql.com/doc/>, 2024.
- [6] Vladimir Agafonkin, *Leaflet.js — an open-source JavaScript library for interactive maps*, <https://leafletjs.com/>, 2024.
- [7] M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, <https://tools.ietf.org/html/rfc7519>, 2015.
- [8] Square, Inc., *Retrofit2 — A type-safe HTTP client for Android and Java*, <https://square.github.io/retrofit/>, 2024.
- [9] Google, *FusedLocationProviderClient — Android Developers*, <https://developers.google.com/location-context/fused-location-provider>, 2024.
- [10] Traccar, *Traccar Open Source GPS Tracking System*, <https://www.traccar.org/>, 2024.
- [11] Gurtam, *Wialon — GPS Fleet Management Platform*, <https://wialon.com/>, 2024.
- [12] Google, *Fleet Engine — Google Maps Platform*, <https://developers.google.com/maps/documentation/transportation-logistics/fleet-engine>, 2024.
- [13] R. Fielding, J. Reschke, *Hypertext Transfer Protocol (HTTP/1.1) : Semantics and Content*, RFC 7231, IETF, <https://tools.ietf.org/html/rfc7231>, 2014.
- [14] Recharts Group, *Recharts — Redefined chart library built with React and D3*, <https://recharts.org/>, 2024.

Annexe A

Code Source — API Backend

A.1 Schéma SQL de la Base de Données

Le script suivant crée les quatre tables de la base de données `fleet_db` avec leurs contraintes d'intégrité référentielle et les index recommandés pour optimiser les requêtes GPS.

```
1  -- Table des chauffeurs
2  CREATE TABLE drivers (
3      id          INT AUTO_INCREMENT PRIMARY KEY,
4      name        VARCHAR(100) NOT NULL,
5      email       VARCHAR(100) NOT NULL UNIQUE,
6      password    VARCHAR(255) NOT NULL,
7      phone       VARCHAR(20),
8      role        ENUM('admin', 'driver') DEFAULT 'driver',
9      status      ENUM('active', 'inactive') DEFAULT 'active',
10     created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
11 );
12
13 -- Table des vehicules
14 CREATE TABLE vehicles (
15     id          INT AUTO_INCREMENT PRIMARY KEY,
16     plate       VARCHAR(20) NOT NULL UNIQUE,
17     brand       VARCHAR(50),
18     model       VARCHAR(50),
19     status      ENUM('active', 'inactive', 'offline') DEFAULT 'offline',
20     driver_id   INT,
21     last_seen   TIMESTAMP NULL,
22     created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
23     FOREIGN KEY (driver_id) REFERENCES drivers(id) ON DELETE SET NULL
24 );
25
26 -- Table des positions GPS
27 CREATE TABLE vehicle_positions (
```

```

28  id          INT AUTO_INCREMENT PRIMARY KEY,
29  vehicle_id  INT NOT NULL,
30  latitude    DECIMAL(10, 8) NOT NULL,
31  longitude   DECIMAL(11, 8) NOT NULL,
32  recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
33  FOREIGN KEY (vehicle_id) REFERENCES vehicles(id) ON DELETE CASCADE
34 );
35
36 -- Table des logs carburant
37 CREATE TABLE fuel_logs (
38  id          INT AUTO_INCREMENT PRIMARY KEY,
39  vehicle_id  INT NOT NULL,
40  driver_id   INT,
41  quantity    DECIMAL(6,2) NOT NULL,
42  note        TEXT,
43  logged_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
44  FOREIGN KEY (vehicle_id) REFERENCES vehicles(id) ON DELETE CASCADE,
45  FOREIGN KEY (driver_id) REFERENCES drivers(id) ON DELETE SET NULL
46 );
47
48 -- Index pour optimiser les requetes GPS et carburant
49 CREATE INDEX idx_positions_vehicle ON vehicle_positions(vehicle_id);
50 CREATE INDEX idx_positions_time    ON vehicle_positions(recorded_at);
51 CREATE INDEX idx_fuellogs_vehicle  ON fuel_logs(vehicle_id);
52 CREATE INDEX idx_fuellogs_driver   ON fuel_logs(driver_id);

```

Listing A.1 – Schéma SQL complet de la base de données

A.2 Middleware d'Authentification JWT

Le middleware suivant vérifie le token JWT à chaque requête protégée. Il extrait le token du header `Authorization`, le vérifie avec la clé secrète, et attache les données de l'utilisateur à l'objet `req` pour les controllers suivants. En cas d'absence ou d'invalidité du token, il retourne respectivement une erreur 401 ou 403.

```

1  const jwt = require('jsonwebtoken');
2
3  module.exports = (req, res, next) => {
4    const authHeader = req.headers['authorization'];
5    // Format attendu : "Bearer <token>"
6    const token = authHeader && authHeader.split(' ')[1];
7
8    if (!token)
9      return res.status(401).json({ message: 'Token manquant' });
10
11    try {

```

```
12     const decoded = jwt.verify(token, process.env.JWT_SECRET);
13     req.user = decoded; // { id, role, name }
14     next();
15   } catch (err) {
16     // Token expire ou signature invalide
17     res.status(403).json({ message: 'Token invalide ou expire' });
18   }
19 };
```

Listing A.2 – Middleware JWT

Annexe B

Manuel d'Installation et de Déploiement

B.1 Prérequis

- Node.js v18 ou supérieur (<https://nodejs.org>)
- XAMPP v8.2 avec MySQL 8.0 (<https://www.apachefriends.org>)
- Android Studio Hedgehog 2023.1.1 (<https://developer.android.com/studio>)
- npm (inclus avec Node.js)
- Git (<https://git-scm.com>)

B.2 Installation du Backend

1. Démarrer XAMPP et activer le service MySQL.
2. Créer la base de données `fleet_db` dans phpMyAdmin.
3. Importer le schéma SQL de l'Annexe A (onglet « Import »).
4. Naviguer vers le dossier `fleet-api`.
5. Installer les dépendances : `npm install`
6. Créer le fichier `.env` avec le contenu suivant :

```
1 DB_HOST=localhost
2 DB_USER=root
3 DB_PASSWORD=
4 DB_NAME=fleet_db
5 JWT_SECRET=votre_cle_secrete_ici    # Remplacer par une valeur forte
6 PORT=3000
```

Listing B.1 – Fichier `.env` du backend

7. Démarrer le serveur : `node server.js`
8. L'API est accessible sur <http://localhost:3000>

B.3 Installation de l'Interface Web

1. Naviguer vers le dossier `fleet-web`.
2. Installer les dépendances : `npm install`
3. Démarrer le serveur de développement : `npx vite`
4. L'interface est accessible sur <http://localhost:5173>

B.4 Installation de l'Application Android

1. Ouvrir le projet `fleet-android` dans Android Studio.
2. Synchroniser Gradle (cliquer sur « Sync Now »).
3. Configurer l'adresse IP dans `ApiClient.java` :
 - **Émulateur Android** : utiliser `http://10.0.2.2:3000/api/`
 - **Appareil physique** : utiliser `http://[IP_LAN_du_PC]:3000/api/`
(trouver l'IP avec `ipconfig` sous Windows ou `ip addr` sous Linux)
4. Lancer l'application sur émulateur ou appareil réel (USB debugging activé).

B.5 Comptes de Test

Le tableau [B.1](#) liste les comptes de test par défaut à créer via l'endpoint `POST /api/auth/register` ou directement en base de données.

TABLE B.1 – Comptes de test par défaut

Rôle	Email	Mot de passe
Admin	admin@fleet.ma	admin123
Chauffeur	mohamed@fleet.ma	driver123

Note de sécurité : ces identifiants sont destinés aux tests locaux uniquement. En production, utiliser des mots de passe forts et ne jamais les exposer dans un dépôt public.

Wissam Oulad Tohami

Fleet Manager : plateforme web et Android pour le suivi intelligent d'une flotte de véhicules

Le présent projet vise à développer un système complet de suivi de flotte de véhicules destiné aux écoles et entreprises. Pour atteindre cet objectif, une approche méthodologique rigoureuse a été adoptée, combinant une analyse fonctionnelle détaillée, une conception modulaire en trois composantes complémentaires, et l'implémentation de technologies modernes du développement web et mobile.

Les contributions majeures incluent la mise en œuvre d'une API REST sécurisée (Node.js/Express), d'une interface web React avec tableau de bord interactif et carte GPS en temps réel (Leaflet), et d'une application Android dédiée aux chauffeurs permettant l'envoi automatique de position GPS et la déclaration de carburant. Ces réalisations ont été validées par des tests Postman sur 16 endpoints et des tests fonctionnels sur émulateur Android.

Ce projet ouvre des perspectives prometteuses : intégration de notifications temps réel via WebSocket, génération de rapports PDF exportables, déploiement cloud, et optimisation du calcul de distances GPS.

Mots clés : suivi de flotte, géolocalisation GPS, API REST, Node.js, React, Android, JWT, MySQL, tableau de bord, carburant.